

ECE 506: Final Exam

Assigned: Saturday, Dec. 6th, 2025.

Due: Submit online by noon, Dec. 7th, 2025.

No collaborations are allowed.
Ask instructor for help if needed.

Problem #1. 2D Linear Programming Problems and Neural Networks

In this problem, we want to modify the Linear Program that we considered in the first homework to consider cases where the ReLU zeros out some of the inputs.

1(a) We begin with the first layer. We want to allow $A_1x_1 + b_1$ to take on any value. However, only the positive part of $A_1x_1 + b_1$ will propagate through the ReLU. The rest will be zero. To do this correctly, like before, define

$$c_1 = c_1^+ - c_1^-, \quad c_1^+, c_1^- \geq 0.$$

We then set $A_1x_1 + b_1 = c_1$. Rewrite $A_1x_1 + b_1 = c_1$ in the form $A'_1y = b'_1$ where y contains the variables:

$$y = [x_1, c_1^-, c_1^+]^T.$$

Give the expressions for A'_1 and b'_1 .

Solution

We introduce the split variable

$$c_1 = c_1^+ - c_1^-, \quad c_1^+, c_1^- \geq 0,$$

and impose the equality

$$A_1x_1 + b_1 = c_1^+ - c_1^-.$$

Rearranging,

$$A_1x_1 - c_1^- + c_1^+ = -b_1.$$

With the variable vector

$$y = [x_1, c_1^-, c_1^+]^T,$$

this becomes a linear system of the form $A'_1y = b'_1$, where

$$A'_1 = \begin{bmatrix} A_1 & -I & I \end{bmatrix}, \quad b'_1 = -b_1.$$

1(b) For the second layer, we follow a similar approach. However, the new input is $x_2 = c_1^+$. We now have a new vector:

$$y = [x_1, c_1^-, c_1^+, c_2^-, c_2^+]^T.$$

Give the expressions for A' and b' so that $A'y = b'$ and your expressions satisfy both 1(a) and

$$A_2c_1^+ + b_2 = c_2.$$

Hint: Your expression for A' will have some zeros. Also, A' will have two rows.

Solution

The second layer uses the ReLU output as its input:

$$x_2 = c_1^+.$$

We again split the output variable:

$$c_2 = c_2^+ - c_2^-, \quad c_2^+, c_2^- \geq 0.$$

The layer-2 equality is

$$A_2 c_1^+ + b_2 = c_2^+ - c_2^-.$$

Rearranging,

$$A_2 c_1^+ - c_2^- + c_2^+ = -b_2.$$

Combining the constraint from 1(a),

$$A_1 x_1 - c_1^- + c_1^+ = -b_1,$$

we obtain two linear equations. Using the variable vector

$$y = [x_1, c_1^-, c_1^+, c_2^-, c_2^+]^T,$$

the system $A'y = b'$ is

$$A' = \begin{bmatrix} A_1 & -I & I & 0 & 0 \\ 0 & 0 & A_2 & -I & I \end{bmatrix}, \quad b' = \begin{bmatrix} -b_1 \\ -b_2 \end{bmatrix}.$$

1(c) Suppose that we have a specific value that we want to achieve for $c_2 \geq 0$. In this case, modify 1(b) by giving new expressions A' , y , and b' so that $A'y = b'$. *Hint: Note that the new y will not contain c_2 as a variable.*

Solution

Now assume that c_2 is no longer a variable but a fixed value satisfying $c_2 \geq 0$. Since

$$c_2 = c_2^+ - c_2^-,$$

and $c_2 \geq 0$, the only feasible representation is

$$c_2^+ = c_2, \quad c_2^- = 0.$$

Thus neither c_2^+ nor c_2^- are variables anymore, and they are removed from y .

The layer-2 equation becomes

$$A_2 c_1^+ + b_2 = c_2,$$

which we rewrite as

$$A_2 c_1^+ = c_2 - b_2.$$

We still keep the equality from part 1(a):

$$A_1 x_1 - c_1^- + c_1^+ = -b_1.$$

Hence the new variable vector is

$$y = [x_1, c_1^-, c_1^+]^T.$$

The system $A'y = b'$ is now

$$A' = \begin{bmatrix} A_1 & -I & I \\ 0 & 0 & A_2 \end{bmatrix}, \quad b' = \begin{bmatrix} -b_1 \\ c_2 - b_2 \end{bmatrix}.$$

Problem #2. Linear models in 2-D Optimization.

The goal of this problem is to make the connection between *max pooling* and optimization. Consider the constrained optimization problem given by:

$$\max_x b^T x + c \quad \text{subject to:} \quad l_1 \leq x_1 \leq u_1 \quad \text{and} \quad l_2 \leq x_2 \leq u_2. \quad (1)$$

2(a) Reformulate the problem as a constrained minimization problem that we studied in class. Then, derive the KKT-based conditions so as to have a unique solution that is at the lower-left corner:

$$x_1 = l_1 \quad \text{and} \quad x_2 = l_2.$$

Recall that for this Linear Program formulation, the solution is always at one of the corners.

Solution

The original problem is

$$\max_x b^T x + c \quad \text{subject to} \quad l_1 \leq x_1 \leq u_1, \quad l_2 \leq x_2 \leq u_2.$$

We convert the maximization to a minimization:

$$\min_x -b^T x - c.$$

Introduce inequality constraints:

$$\begin{aligned} g_1(x) &= x_1 - u_1 \leq 0, & g_2(x) &= l_1 - x_1 \leq 0, \\ g_3(x) &= x_2 - u_2 \leq 0, & g_4(x) &= l_2 - x_2 \leq 0. \end{aligned}$$

The Lagrangian is

$$L(x, \lambda) = -b^T x - c + \lambda_1(x_1 - u_1) + \lambda_2(l_1 - x_1) + \lambda_3(x_2 - u_2) + \lambda_4(l_2 - x_2),$$

with $\lambda_i \geq 0$.

KKT Stationarity

$$\nabla_x L = -b + \begin{bmatrix} \lambda_1 - \lambda_2 \\ \lambda_3 - \lambda_4 \end{bmatrix} = 0.$$

Thus,

$$\lambda_1 - \lambda_2 = b_1, \quad \lambda_3 - \lambda_4 = b_2.$$

Conditions for the lower-left corner At the lower-left corner,

$$x_1 = l_1, \quad x_2 = l_2,$$

the active constraints are:

$$g_2(x) = 0 \Rightarrow \lambda_2 > 0, \quad g_4(x) = 0 \Rightarrow \lambda_4 > 0,$$

and the inactive ones have

$$\lambda_1 = 0, \quad \lambda_3 = 0.$$

Plugging into stationarity gives

$$0 - \lambda_2 = b_1 \Rightarrow \lambda_2 = -b_1 > 0,$$

$$0 - \lambda_4 = b_2 \Rightarrow \lambda_4 = -b_2 > 0.$$

Therefore, the conditions for a unique solution at the lower-left corner are:

$$b_1 < 0, \quad b_2 < 0.$$

If both components of b are negative, the objective $b^T x$ is minimized at the smallest feasible values of x_1 and x_2 , giving the unique optimal point:

$$x^* = (l_1, l_2).$$

2(b) Assume that we have the same function as in 2(a). However, we consider a new bounding box. The new problem is:

$$\max_x b^T x + c \quad \text{subject to:} \quad l'_1 \leq x_1 \leq u'_1 \quad \text{and} \quad l'_2 \leq x_2 \leq u'_2. \quad (2)$$

Assume that the new bounding box of 2(b) contains the bounding box of 2(a) as given by:

$$l'_1 \leq l_1, \quad l'_2 \leq l_2, \quad u'_1 \geq u_1, \quad u'_2 \geq u_2. \quad (3)$$

Assume that the maximum lies in the lower-left corner for 2(a). Will the new maximum also be at the new lower-left corner for the new box? Do you need any additional conditions to guarantee that the maximum remains at the new lower-left corner (or not)?

Solution

From 2(a), the maximization

$$\max_x b^T x + c$$

achieves its optimum at the lower-left corner (l_1, l_2) exactly when

$$b_1 < 0, \quad b_2 < 0.$$

Now consider the enlarged bounding box satisfying

$$l'_1 \leq l_1, \quad l'_2 \leq l_2, \quad u'_1 \geq u_1, \quad u'_2 \geq u_2.$$

The smallest feasible point of the new box is (l'_1, l'_2) .

Since $b_1 < 0$ and $b_2 < 0$, the linear objective

$$b^T x = b_1 x_1 + b_2 x_2$$

is maximized by choosing each component of x as small as possible. Therefore,

$$x_1^* = l'_1, \quad x_2^* = l'_2$$

is the maximizing point of the new problem.

No additional conditions are required: the same sign conditions $b_1 < 0$ and $b_2 < 0$ that forced the optimum to (l_1, l_2) in 2(a) automatically force the optimum to the new lower-left corner (l'_1, l'_2) of the expanded box.

Problem 3. [Stepsizes and convergence rates for line search methods]**3(a)** Consider the optimal step-size derived for steepest-descent for a quadratic:

$$f(x) = \frac{1}{2}x^T Qx - b^T x$$

$$\alpha_k = \frac{\nabla f_k^T \nabla f_k}{\nabla f_k^T Q \nabla f_k}$$

$$x_{k+1} = x_k - \alpha_k \nabla f_k$$

Now, assume that the gradient is along an eigenvector direction:

$$\nabla f_k = a e_p, \quad Q e_p = \lambda_p e_p.$$

Derive a simplified expression for α_k . Assume that the eigenvector is a unit vector $e_p^T e_p = 1$.**Solution**

We are given

$$\alpha_k = \frac{\nabla f_k^T \nabla f_k}{\nabla f_k^T Q \nabla f_k}, \quad \nabla f_k = a e_p, \quad Q e_p = \lambda_p e_p, \quad e_p^T e_p = 1.$$

Compute the numerator:

$$\nabla f_k^T \nabla f_k = (a e_p)^T (a e_p) = a^2 (e_p^T e_p) = a^2.$$

Compute the denominator:

$$\nabla f_k^T Q \nabla f_k = (a e_p)^T Q (a e_p) = a^2 e_p^T (Q e_p) = a^2 e_p^T (\lambda_p e_p) = a^2 \lambda_p.$$

Thus,

$$\alpha_k = \frac{a^2}{a^2 \lambda_p} = \frac{1}{\lambda_p}.$$

So the optimal step-size in the eigenvector direction is

$$\alpha_k = \frac{1}{\lambda_p}.$$

3(b) What is the minimum possible value for α_k ?**Solution**

From part (a), the optimal step-size is

$$\alpha_k = \frac{1}{\lambda_p},$$

where λ_p is the eigenvalue corresponding to the gradient direction.

Since $\alpha_k = 1/\lambda_p$, the minimum possible value occurs when the eigenvalue is as large as possible:

$$\alpha_k^{\min} = \frac{1}{\lambda_{\max}}.$$

3(c) What is the maximum possible value for α_k ?

Solution

From part (a), the optimal step-size is

$$\alpha_k = \frac{1}{\lambda_p}.$$

The maximum value occurs when the eigenvalue λ_p is as small as possible. Thus,

$$\alpha_k^{\max} = \frac{1}{\lambda_{\min}}.$$

3(d) What is x_{k+1} when all of the eigenvalues are equal?

Solution

If all eigenvalues of Q are equal, say

$$\lambda_1 = \lambda_2 = \cdots = \lambda = \lambda_p,$$

then from part (a) the step-size is the same in every direction:

$$\alpha_k = \frac{1}{\lambda}.$$

The steepest-descent update is

$$x_{k+1} = x_k - \alpha_k \nabla f_k = x_k - \frac{1}{\lambda} \nabla f_k.$$

Since the Hessian is proportional to the identity, steepest descent behaves like gradient descent on a spherical quadratic, and the update becomes

$$x_{k+1} = x_k - \frac{1}{\lambda} \nabla f_k.$$

Problem #4. Multiobjective Optimization.

For this problem, note that all of the points use integer coordinate points. For the solutions, make sure to list the coordinates of the original points. If you want, you can simply circle the Pareto-optimal points for each part. I plotted objective2 along the x-axis and objective1 along the y-axis. When minimizing, focus on the objectives being minimized, not the order in which they appear. I have removed the parentheses from the pairs.

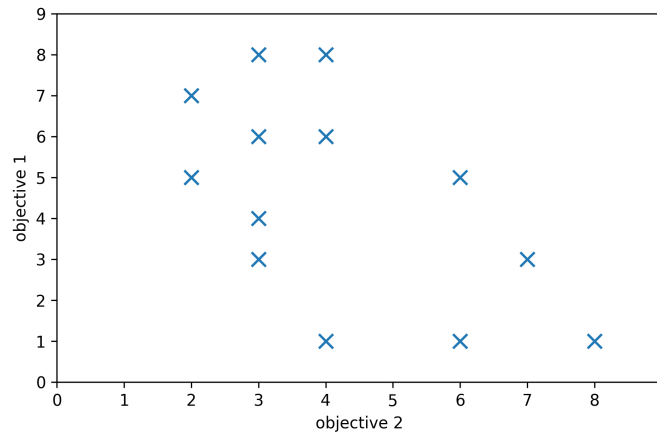


Figure 1: Figure for problem #4.

4(a) List the Pareto points for the following problem:

$$\min \text{objective1, objective2} \quad (4)$$

Solution

A point is Pareto-optimal if no other point has both a smaller value of objective1 and a smaller value of objective2.

Evaluating the given points, the nondominated points (the southwest boundary of the plot) are:

$$(2, 5), \quad (3, 3), \quad (4, 1).$$

Thus, the Pareto set is

$$\{(2, 5), (3, 3), (4, 1)\}$$

4(b) List the Pareto points for the following problem:

$$\min -\text{objective1, objective2} \quad (5)$$

Solution

For this case we minimize objective2 while maximizing objective1 (since $-\text{objective1}$ is minimized).

A point is Pareto-optimal if no other point has both

larger objective1 and smaller objective2.

Using the given points:

$(2, 5), (2, 7), (3, 3), (3, 4), (3, 6), (3, 8), (4, 1), (4, 6), (4, 8), (6, 1), (6, 5), (7, 3), (8, 1),$

we examine each objective2 value and keep the point with the *largest* objective1 that is not dominated.

This yields the nondominated set:

$(4, 1), (7, 3), (3, 8).$

These points form the Pareto frontier when maximizing objective1 and minimizing objective2.

4(c) List the Pareto points for the following problem:

$$\min \text{objective1}, -\text{objective2} \quad (6)$$

Solution

A point is Pareto-optimal if no other point has both

smaller objective1 and larger objective2.

Using the given points, we scan from the lowest objective1 upward while keeping only those with the largest possible objective2 at each feasible level.

This produces the nondominated set:

$(4, 1), (3, 3), (2, 7).$

These points form the Pareto frontier when minimizing objective1 and maximizing objective2.

4(d) List the Pareto points for the following problem:

$$\min -\text{objective1}, -\text{objective2} \quad (7)$$

Solution

We seek points that are not dominated by any point with both larger objective1 and larger objective2.

Scanning the point set, the nondominated points under simultaneous maximization of both objectives are:

$(3, 8), (4, 8).$

These points lie on the northeast boundary of the plot and cannot be dominated in both objectives.

Problem #5. Validation and loss functions.

For each one of the following graphs, answer the following questions:

- Is the model overfitting? Why? Or why not?
- Is the model underfitting? Why? Or why not?
- Do we get the optimal model for the last epoch? Why? Or why not?

5(a) Refer to Fig. 5(a).

Solution

- **Overfitting?** Yes. The training loss continues to decrease, but the validation loss reaches a minimum and then increases afterward. This divergence indicates the model is overfitting after the early epochs.
- **Underfitting?** No. The training loss becomes very small, showing that the model fits the training data well. Underfitting would require both losses to remain high.
- **Optimal model at the last epoch?** No. The validation loss is higher at the last epoch than it was earlier. The optimal model occurs at the epoch where the validation loss is minimized (around epoch 3–4), not at the final epoch.

5(b) Refer to Fig. 5(b).

Solution

- **Overfitting?** No. The validation loss closely follows the training loss and continues to decrease. There is no divergence between the two curves, so the model is not overfitting.
- **Underfitting?** No. Both losses decrease steadily and reach relatively low values. If the model were underfitting, the losses would remain high or flat.
- **Optimal model at the last epoch?** Yes. Since the validation loss continues to decrease and does not show any upward trend, the final epoch corresponds to the lowest validation loss and is therefore the optimal model.

5(c) Refer to Fig. 5(c).

Solution

- **Overfitting?** Partially. The validation loss initially increases around epochs 2–3 while the training loss continues to decrease. This indicates temporary overfitting. However, the validation loss later decreases again, reducing the overfitting effect.

- **Underfitting?** No. The training loss becomes very small, and the validation loss eventually decreases as well. Underfitting would require both losses to remain high.
- **Optimal model at the last epoch?** Yes. By the final epoch, the validation loss is near its lowest value and close to the training loss. Since the validation loss improves again after the early overfitting phase, the last epoch gives the best-performing model.

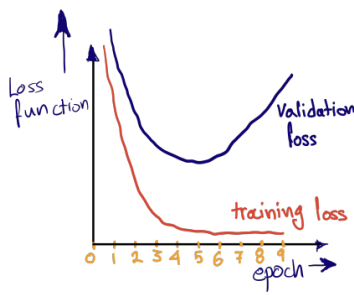


Figure 2: *
Figure 2: Figure for problem 5(a).



Figure 3: *
Figure 3: Figure for problem 5(b).



Figure 4: *
Figure 4: Figure for problem 5(c).

Problem 6. Convex and concave functions without computing the Hessians.

While most optimization methods are based on convexity, we want to consider methods that cover the general case of monotone functions where the Hessian is too expensive to obtain. This is clearly the case for AI and Machine Learning problems.

6(a) [Convex and concave functions] A convex function lies above its tangents:

$$f(y) \geq f(x) + \nabla f(x)^T(y - x).$$

We also know that convex functions lie below the line segment that connects any two points:

$$f(\theta x_k + (1 - \theta)x_{k+1}) \leq \theta f(x_k) + (1 - \theta)f(x_{k+1}) \quad \text{for } 0 \leq \theta \leq 1.$$

Suppose that f represents a non-negative loss function: $f(x) \geq 0$. Consider the line segment that connects x_k and x_{k+1} and the line segment that connects $f(x_k)$ and $f(x_{k+1})$. Apply the three inequalities to derive a region where f must lie between x_k and x_{k+1} . You need to sketch all possible regions to receive full credit for this problem.

Solution

A convex, nonnegative function on the interval $[x_k, x_{k+1}]$ is restricted by three inequalities:

1. Tangential lower bound

$$f(y) \geq f(x_k) + \nabla f(x_k)^T(y - x_k),$$

so f must lie above the tangent line at x_k .

2. Secant upper bound

$$f(\theta x_k + (1 - \theta)x_{k+1}) \leq \theta f(x_k) + (1 - \theta)f(x_{k+1}),$$

so f must lie below the straight line connecting the two endpoints.

3. Nonnegativity bound

$$f(y) \geq 0,$$

so f must also lie above the horizontal axis.

Combining these three inequalities, f is restricted to lie in the region

$$\max\{0, \text{tangent at } x_k\} \leq f(y) \leq \text{secant line between } (x_k, f(x_k)) \text{ and } (x_{k+1}, f(x_{k+1})).$$

This region is illustrated in Fig. 6(a).

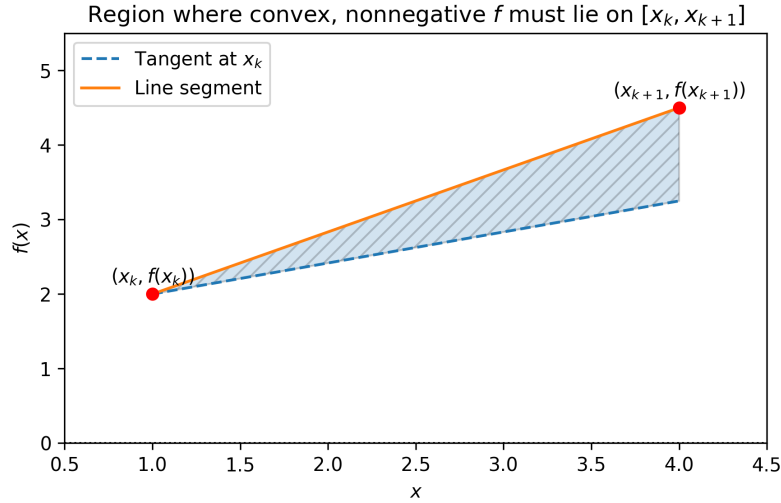


Figure 5: Region where a convex, nonnegative function must lie between x_k and x_{k+1} .

6(b) Repeat 6(a) for a concave function with the following two bounds:

$$f(x) \geq 0 \quad \text{and} \quad f(x) \leq U.$$

Again, you need to sketch all possibilities to receive full credit.

Solution

For a concave function, the inequalities reverse compared to the convex case. Between x_k and x_{k+1} :

1. Tangential upper bound

$$f(y) \leq f(x_k) + \nabla f(x_k)^T (y - x_k),$$

so the function must lie *below* the tangent line at x_k .

2. Secant lower bound

$$f(\theta x_k + (1 - \theta)x_{k+1}) \geq \theta f(x_k) + (1 - \theta)f(x_{k+1}),$$

so f must lie *above* the line segment connecting the two endpoint values.

3. Given bounds

$$0 \leq f(y) \leq U,$$

so the function is also constrained between the horizontal lines $f = 0$ and $f = U$.

Combining these inequalities, a concave, nonnegative, upper-bounded function must satisfy

$$\text{secant line} \leq f(y) \leq \min\{U, \text{tangent at } x_k\}.$$

This creates a feasible region *between the secant line and the tangent line*, but also clipped between 0 and U . Figure 6(b) illustrates all possible shapes f may take.

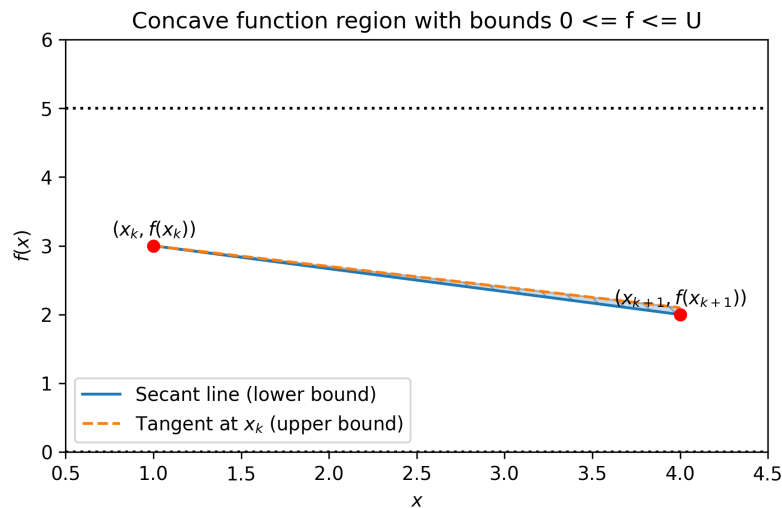


Figure 6: Region where a concave function must lie under bounds $0 \leq f \leq U$.

6(c) Suppose that we have the function and its gradient at two points:

$$f_k, \nabla f_k, f_{k+1}, \nabla f_{k+1}.$$

Additionally, assume that you have stored $x_k, x_{k+1}, \dots, x_{k+m}$. Provide the pseudocode for detecting whether the function is convex or concave.

Solution

To test convexity or concavity on $[x_k, x_{k+1}]$, compare the directional derivatives along

$$d = x_{k+1} - x_k, \quad \phi_k = \nabla f_k^T d, \quad \phi_{k+1} = \nabla f_{k+1}^T d.$$

A convex function has increasing directional derivatives; a concave function has decreasing ones.

```

d      = x_{k+1} - x_k
phi_k  = grad_f_k^T d
phi_k1 = grad_f_{k+1}^T d

if phi_k1 > phi_k:
    function is convex
elif phi_k1 < phi_k:
    function is concave
else:
    function is locally affine

```

Apply this test to each stored interval $[x_i, x_{i+1}]$ for $i = k, \dots, k + m - 1$.

6(d) Suppose that you have the function and its gradient at multiple points:

$$f_k, \nabla f_k, f_{k+1}, \nabla f_{k+1}, \dots, f_{k+m}, \nabla f_{k+m}.$$

Provide the pseudocode that detects that your function is concave for m evaluations.

Solution

For concavity, the directional derivatives along each segment

$$d_i = x_{i+1} - x_i$$

must be nonincreasing:

$$\phi_i = \nabla f_i^T d_i, \quad \phi_{i+1} \leq \phi_i \quad \forall i.$$

Pseudocode:

```
is_concave = True

for i = k to k + m - 1:
    d_i = x_{i+1} - x_i
    phi_i = grad_f_i^T d_i
    phi_ip = grad_f_{i+1}^T d_i

    if phi_ip > phi_i:
        is_concave = False
        break

if is_concave:
    print("Function is concave on [x_k, x_{k+m}]")
else:
    print("Function is not concave")
```

6(e) Repeat 6(d) for detecting that your function is convex for m evaluations.

Solution

For convexity, the directional derivatives along each segment

$$d_i = x_{i+1} - x_i$$

must be nondecreasing:

$$\phi_i = \nabla f_i^T d_i, \quad \phi_{i+1} \geq \phi_i \quad \forall i.$$

Pseudocode:


```

is_convex = True

for i = k to k + m - 1:
    d_i      = x_{i+1} - x_i
    phi_i    = grad_f_i^T    d_i
    phi_ip   = grad_f_{i+1}^T d_i

    if phi_ip < phi_i:
        is_convex = False
        break

if is_convex:
    print("Function is convex on [x_k, x_{k+m}]")
else:
    print("Function is not convex")

```

6(f) Based on your answers in 6(d) and 6(e), determine when a function is neither convex nor concave within a region that contains

$$x_k, x_{k+1}, \dots, x_{k+m}.$$

Solution

From 6(d) and 6(e), convexity requires the directional derivatives

$$\phi_i = \nabla f_i^T (x_{i+1} - x_i)$$

to be nondecreasing:

$$\phi_{i+1} \geq \phi_i,$$

and concavity requires them to be nonincreasing:

$$\phi_{i+1} \leq \phi_i.$$

A function is therefore *neither* convex nor concave on the region containing $x_k, x_{k+1}, \dots, x_{k+m}$ when the sequence $\{\phi_i\}$ violates both conditions; i.e.,

$$\exists i, j \text{ such that } \phi_{i+1} < \phi_i \text{ and } \phi_{j+1} > \phi_j.$$

In words: if the directional derivatives increase on some intervals and decrease on others, the function cannot be convex or concave over the entire region.

Problem #7. This problem is based on *ADAM: A Method for Stochastic Optimization* that can be accessed at <https://arxiv.org/pdf/1412.6980>. Additionally, consult with the `ML-intro.pdf` document.

7(a) Derive an update expression for the sample covariance of the gradient vector. For this problem, assume that you already have update equations for the sample mean of the gradient. Your goal is to derive an expression for S_{t+1} in terms of S_t and g_{t+1} .

Solution

Assume the sample mean of the gradients is updated by

$$m_{t+1} = (1 - \beta)m_t + \beta g_{t+1}.$$

The sample covariance may be updated using the standard online second-moment formula:

$$S_{t+1} = (1 - \beta)S_t + \beta(g_{t+1} - m_t)(g_{t+1} - m_{t+1})^T.$$

Since

$$g_{t+1} - m_{t+1} = (1 - \beta)(g_{t+1} - m_t),$$

we obtain the simplified update:

$$S_{t+1} = (1 - \beta)S_t + \beta(1 - \beta)(g_{t+1} - m_t)(g_{t+1} - m_t)^T.$$

7(b) Give an expression for the unbiased estimate of the covariance matrix Σ in terms of S_{t+1} .

Solution

Because S_{t+1} is an exponential moving average initialized at $S_0 = 0$, it is biased toward 0. As in ADAM, the unbiased estimator is obtained by dividing out the missing mass of the exponential decay:

$$\hat{\Sigma}_{t+1} = \frac{S_{t+1}}{1 - \beta^{t+1}}.$$

7(c) We want to modify S_{t+1} to make it invertible. This is going to be a generalization of the ADAM method. Note that the sample covariance matrix and Σ are symmetric positive semi-definite matrices that can be expressed as $Q\Lambda Q^T$ where Q is the eigenvector matrix and Λ is a diagonal matrix with non-negative eigenvalues, and $QQ^T = Q^TQ = I$. Consider modifying S_{t+1} so as to have positive eigenvalues using:

$$S' = S_{t+1} + \epsilon I.$$

Derive an expression for ϵ in terms of the smallest eigenvalue of S_{t+1} so that S' is invertible.

Solution

Let the eigenvalue decomposition of S_{t+1} be

$$S_{t+1} = Q\Lambda Q^T, \quad \Lambda = \text{diag}(\lambda_1, \dots, \lambda_d),$$

where each $\lambda_i \geq 0$ and $\lambda_{\min} = \min_i \lambda_i$.

After adding a multiple of the identity,

$$S' = S_{t+1} + \epsilon I,$$

the eigenvalues shift to

$$\lambda'_i = \lambda_i + \epsilon.$$

For S' to be invertible, all eigenvalues must be strictly positive:

$$\lambda_{\min} + \epsilon > 0.$$

Thus the required condition on ϵ is

$$\epsilon > -\lambda_{\min}.$$

Since $\lambda_{\min} \geq 0$ for a covariance matrix, it suffices to choose

$$\epsilon > 0.$$

7(d) We want to modify the normalization step using the unbiased estimate of the covariance matrix. Recall the following expression:

$$\text{Cov}(AX) = A \text{Cov}(X) A^T.$$

Derive an expression for A so that $AS' = I$.

Solution

We want a matrix A such that

$$AS' = I, \quad S' = S_{t+1} + \epsilon I.$$

Since S' is symmetric positive definite, it has the eigendecomposition

$$S' = Q\Lambda Q^T,$$

where $\Lambda = \text{diag}(\lambda'_1, \dots, \lambda'_d)$ with $\lambda'_i > 0$.

For $AS' = I$, we require

$$A = (S')^{-1}.$$

Using the eigen-decomposition,

$$(S')^{-1} = Q\Lambda^{-1}Q^T,$$

which gives the desired normalization matrix

$$A = Q\Lambda^{-1}Q^T.$$

7(e) Going back to the original ADAM algorithm, what is the assumed shape for S' ?

Solution

In the original ADAM algorithm, S' corresponds to the second-moment accumulator v_t , which stores only the elementwise variances of the gradient. Thus S' is assumed to be a *diagonal matrix*. Equivalently, ADAM treats the covariance as

$$S' = \text{diag}(v_t),$$

so no cross-correlations between gradient components are modeled.